

Phase 1 — Data Cleaning Plan

Problems found in each data file, the evidence, the agreed solution and how each fix will be checked.

Author	Tanay Nagpal
Date	30 September 2026
Status	✓ All 6 files planned · 51 problems agreed
Data	Fitbit tracker data · 33 people · 12 Apr – 12 May 2016

Contents

- 0. Summary
- 1. File 1 · dailyActivity
- 2. File 2 · Hourly (steps, calories, intensities)
- 3. File 3 · sleepDay
- 4. File 4 · minuteSleep
- 5. File 5 · weightLogInfo
- 6. File 6 · heartrate_seconds

00 · Phase 1 Cleaning Plan — Summary

Project: Strava Fitness Data Analytics · **Author:** Tanay Nagpal · **Phase:** 1 (Planning) · **Date:** 30 Sep 2026
Status: ✓ **ALL 6 FILES PLANNED** — 51 problems agreed

1 · Business problem & objective (refined from the case study to fit the data)

Business problem. Strava collects detailed tracker data on users' daily activity, sleep and body metrics, but has no clear view of **which kinds of users it has, how their activity and sleep habits differ, and which tracking features they actually use**. Without this, marketing and engagement campaigns stay one-size-fits-all instead of targeted by segment, timed to user behaviour, and focused on under-used features.

Business objective. Analyse activity, sleep and body-metric data to **segment users by activity level and tracking consistency**, profile each segment's daily/hourly activity, sleep behaviour and feature adoption, and turn the differences into **data-driven marketing recommendations** — *which segment to target, with what message, promoting which feature, at what time* — presented in a Streamlit dashboard.

Important context. The data is a public Fitbit tracker dataset: 33 people, 12 Apr – 12 May 2016. It has **no activity types** (no running/cycling labels) — effort levels, distance and step-free effort are used instead. Findings are directional (small sample), and correlation is never presented as cause.

2 · Files used and skipped

Used (6 groups)	Why
dailyActivity	Main table — one row per person per day
hourlySteps + hourlyCalories + hourlyIntensities	Time-of-day patterns
sleepDay	Nightly sleep totals
minuteSleep	Bedtime, wake time, naps, restlessness
weightLogInfo	BMI profile + feature adoption
heartrate_seconds	Supporting evidence (effort, data quality)

Skipped (10 files)	Why
dailyCalories, dailySteps, dailyIntensities	100% identical to dailyActivity
minuteSteps / minuteCalories / minuteIntensities (Narrow + Wide, 6 files)	Add up exactly to the hourly files; Wide = same data re-arranged
minuteMETsNarrow	Low priority; effort already covered by intensity

3 · Project-wide rules (apply to every file)

#	Rule
1	Dates: always convert with the explicit American format (month/day/year, AM/PM) — never let Python guess (Indian day/month trap)
2	Remove only the impossible , never the unusual — high values from real workouts stay
3	Never invent data — no filling missing days/hours; use blank (not 0) when a value is unknown
4	Record before removing — <code>wear_log</code> saves every removed day for the engagement page
5	Valid day = not the person's last day · ≥ 100 steps · movement in ≥ 10 different hours
6	7-day rule — per-person profiles need ≥ 7 valid days / reliable nights
7	Medians for "typical" values (one special day can't pull them)

#	Rule
8	One table = one level of detail — person-level items go to <code>user_profile</code>
9	Main activity measures = steps + active minutes ; sitting time only on complete sleep nights
10	State the sample size on every chart that uses a subset (sleep, weight, heart rate)
11	Neutral wording — no health or medical labels; correlation $\neq$ cause

4 · Clean outputs (what Phase 2 will produce)

Output file	Rows	One row =	Built from
<code>daily_clean.csv</code>	757	person-day	<code>dailyActivity</code> (+ movement hours, sleep flags)
<code>wear_log.csv</code>	1,023	person × calendar day	<code>dailyActivity</code> (worn 757 · not worn 77 · partial 73 · last day 33 · stopped 83)
<code>hourly_clean.csv</code>	17,987	person-hour	3 hourly files (750 valid days)
<code>sleep_nights.csv</code>	410 (384 reliable)	person-night	<code>minuteSleep</code> + <code>sleepDay</code>
<code>weight_logs_clean.csv</code>	67	weight entry	<code>weightLogInfo</code>
<code>user_body_profile.csv</code>	8	person	<code>weightLogInfo</code>
<code>daily_heart_rate.csv</code>	280	person-day	<code>heartrate_seconds</code>
<code>user_profile.csv</code>	33	person	all of the above (built in feature engineering)

5 · Problems per file

File	Problems	Rows: raw → clean
01 · <code>dailyActivity</code>	11 (P1-P11)	940 → 757
02 · Hourly (3 files)	9 (H1-H9)	22,099 → 17,987
03 · <code>sleepDay</code>	9 (S1-S9)	413 → 410 nights
04 · <code>minuteSleep</code>	8 (M1-M8)	188,521 minutes → 459 sessions → 410 nights
05 · <code>weightLogInfo</code>	7 (W1-W7)	67 → 67 entries + 8 profiles
06 · <code>heartrate_seconds</code>	7 (R1-R7)	2,483,658 readings → 280 days
Total	51	

6 · Key flags created

Table	Flags
<code>daily_clean</code>	<code>sleep_tracked</code> (365 days), <code>breakdown_missing</code> (7), <code>logged_workout</code> (31), <code>is_weekend</code>
<code>hourly_clean</code>	<code>stepless_effort</code> (18 hours)
<code>sleep_nights</code>	<code>reliable_night</code> (384), <code>incomplete_night</code> (22), <code>capped_session</code> (4), <code>long_sleep</code> (10), <code>took_nap</code>
<code>user_profile</code>	<code>low_data_user</code> (2), <code>low_sleep_data_user</code> (8), <code>low_hr_data_user</code> (2), <code>restless_sleeper</code> (1), <code>stopped_early</code> (4), <code>stopped_final_week</code> (8), <code>logs_weight</code> (8), <code>logs_workouts</code> (3), <code>sleep_adoption_level</code> , <code>features_used</code> , <code>clean_day_share</code> , <code>non_wear_days</code>

7 · How the files depend on each other

This...	...needs this first	Why
<code>daily P4 (movement_hours)</code>	Hourly files joined	Hours with ≥ 1 step
<code>wear_log</code>	Raw <code>dailyActivity</code>	Must be built before any removal

This...	...needs this first	Why
daily P5 (sleep_tracked)	sleep_nights (S5, S6)	Only complete, reliable nights count (365 days)
Hourly H3 / H8	daily_clean	Same valid days everywhere
Heart rate R5	daily_clean	Same valid days everywhere
sleepDay S4 / S9	minuteSleep sessions	Naps vs night, bedtime
user_profile	Everything	Built last

8 · Build order for Phase 2

Step	Action
1	Load all 6 sources; convert dates with explicit formats
2	Join the 3 hourly files; compute movement_hours per person-day
3	Build wear_log from the raw daily rows
4	Clean dailyActivity (P2 → P3 → P4 → P7 → P11)
5	minuteSleep → sessions → nights (naps, pieces, flags) → sleep_nights
6	Add sleep flags to daily (sleep_tracked , sedentary_awake_min)
7	Filter hourly to valid days → hourly_clean
8	Weight → weight_logs_clean + user_body_profile
9	Heart rate → daily_heart_rate
10	Build user_profile ; run every "How we'll check" test

9 · Findings already spotted during cleaning (to confirm in EDA)

- **Non-wear is concentrated:** 12 of 33 people leave the band off; strongest pattern = take it off during the day (e.g., to charge) and forget.
- **Novelty fades:** non-wear rises from 6.6% (week 1) to 9–10% (weeks 2–4); 4 clear drop-outs.
- **Peak activity time:** 5–7 PM (6 PM highest); wake ≈ 7 AM, bed ≈ 11:15 PM.
- **Step counts miss non-walking workouts:** one user ranks 16th by steps but 4th by hard-effort minutes.
- **Sleep:** typical night 7 h 13 min, 94% efficiency; 22% of reliable nights under 6 h; weekend lie-ins and recovery sleep after very active days.
- **Feature depth:** people using more optional features (sleep, weight, workout logging) tend to be more active and wear the band more.
- **Adoption gaps:** sleep tracking used consistently by ~half; weight logging by 24% (2 habitual); workout logging by 9%.

01 · dailyActivity — Cleaning Notes

Project: Strava Fitness Data Analytics · **Phase:** 1 (Planning) · **Role:** MAIN file · **Status:** ✓ ALL 11 PROBLEMS AGREED — plan complete

This is a running log. Each problem we find is written down with its evidence, why it matters, the solution, and how we will check the fix worked. Status shows whether we have agreed on it.

File at a glance

Item	Value
What it is	One row = one person, one day. Steps, distance, minutes at each effort level, calories
Size	940 rows × 15 columns
People	33
Dates	12 Apr 2016 → 12 May 2016 (31 days)
Blanks / duplicate rows / negative values	None / None / None
Replaces	dailyCalories, dailySteps, dailyIntensities (100% identical data — not used)

Columns we keep / drop

Keep	Drop (and why)
Id, ActivityDate → date, TotalSteps, TotalDistance, VeryActiveMinutes, FairlyActiveMinutes, LightlyActiveMinutes, SedentaryMinutes, Calories, LoggedActivitiesDistance, VeryActiveDistance, ModeratelyActiveDistance, LightActiveDistance	TrackerDistance — same as TotalDistance except on hand-logged workout days · SedentaryActiveDistance — almost always 0 (max 0.11 km)

Problem summary

#	Problem	Solution in one line	Rows affected	Status
P1	Date stored as text	Convert to real date using month/day/year	all	✓ Agreed
P2	Incomplete last days	Remove each person's final recorded day	−33	✓ Agreed
P3	Band not worn (zero / near-zero step days)	Count them per user, then remove days with < 100 steps	−77	✓ Agreed
P4	Band worn only part of the day	Remove days with movement in fewer than 10 different hours	−73	✓ Agreed
P5	Sitting time recorded two ways	Flag <code>sleep_tracked</code> ; create <code>sedentary_awake_minutes</code> (sleep-tracked days only); active minutes = main measure	0 (new columns)	✓ Agreed
P6	TotalDistance ≠ TrackerDistance	Use TotalDistance; drop TrackerDistance; turn logged distance into a workout-logging flag	0	✓ Agreed
P7	Effort-level breakdown missing on some days	Blank the breakdown on 7 days (flag <code>breakdown_missing</code>); keep their steps/distance/calories	0 (7 days blanked)	✓ Agreed
P8	Very high / very low values (possible outliers)	Keep all (none impossible); use median for each person's typical day	0	✓ Agreed
P9	Some users have very few clean days	Flag users with < 7 clean days: out of activity profiles/segments, in engagement analysis	0 (flag)	✓ Agreed

#	Problem	Solution in one line	Rows affected	Status
P10	Missing calendar days (people who stopped)	Never fill; mark stopped in wear_log; drop-out flags; trend charts show users-per-week	0	✓ Agreed
P11	Messy / unneeded columns	Drop 2, rename all to simple names, fix types, add helper columns; fixed cleaning order	0	✓ Agreed

Expected result: 940 → 757 rows, still 33 people.

P1 · Date stored as text — ✓ Agreed

What's wrong `ActivityDate` looks like a date ("4/12/2016") but Python reads it as plain text, like a word.

Evidence

- Text sorting puts 5/10/2016 before 5/2/2016 (it compares letter by letter).
- Joining `dailyActivity` with `sleepDay` on the text dates matched **0 rows**, because `sleepDay` writes the same day as 4/12/2016 12:00:00 AM. After converting both to real dates: **410 matches**.
- The dates are American format (**month/day/year**). Reading them the Indian way (day/month/year) turns 4/12/2016 into **4 December** and fails on **578 of 940 rows** ("month 13" doesn't exist).

Why it matters Wrong order in trend charts, no weekday/weekend analysis, and a silent failure when joining with sleep data.

Solution

- Convert `ActivityDate` to a real date, stating the format explicitly: month/day/year.
- Rename it to `date` (same name in every file → easy joins).

How we'll check

- 0 conversion failures
- Earliest date = 12 Apr 2016, latest = 12 May 2016
- 12 Apr 2016 shows as **Tuesday**
- Join with `sleepDay` returns 410 matches

P2 · Incomplete last days — ✓ Agreed

What's wrong The data was exported partway through **12 May 2016** (nothing recorded after ~3 PM). Also, **12 people stopped wearing the band before 12 May**, and their own last day is incomplete too (the day they took it off).

Evidence

Middle value	Normal day	12 May
Steps	8,148	3,121
Calories	2,227	1,212
Active minutes	258	71
Minutes recorded	1,364	910

Examples of early-stoppers' last days: user 4057192912 on 15 Apr (970 min recorded), 3372868164 on 1 May (1,014 min), 6775888955 on 7 May (607 min), 1644430081 on 11 May (762 min). On their other days these users record the full 1,440 minutes.

Why it matters Keeping them creates a false "drop" at the end of the month in trend charts → a wrong story that "users lost interest". It also lowers everyone's averages.

Solution Remove **each person's final recorded day** (12 May for 21 people; the stop day for the other 12). Never scale a partial day up — that would be inventing data.

How we'll check

- No 12 May rows remain
- Each person has exactly one fewer day
- Still 33 people
- Month trend line no longer dips at the end

Rows removed: 33 (940 → 907)

P3 · Band not worn (zero / near-zero step days) — ✓ Agreed (option B)

What's wrong Some days show 0 steps and "sitting for 1,440 minutes" — a whole day without a single movement. That is not a lazy person; it is a day with **no wear data**. A few more days show a tiny number of steps (4, 8, 16, 44...) all inside **one single hour**, and nothing else all day — the band was briefly handled, not worn.

Evidence (after P2)

- **69 days with 0 steps** — 68 of them are exactly 1,440 sitting minutes; together they contain only 33 active minutes in total.
- **8 more days with 1-100 steps**, all with 1,366–1,440 minutes "recorded" (e.g., user 1844505072: 8, 4 and 44 steps on three different days).
- Even a sick person at home walks a few hundred steps a day (bathroom, kitchen) — under 100 steps in 24 hours is not a real day of wear.
- **Calories trap:** these days still show ~1,841 calories (median). The device fills in an estimate of resting calories whether or not it's worn — so calories alone can't tell us the band was worn.
- **It's concentrated:** 12 users have zero-step days. User 4020332650 didn't wear the band on **47%** of their days, 1927972279 on **43%**, 6775888955 on **36%**.

Why it matters

- Average steps per day: **7,814 with these days vs 8,457 without** — keeping them understates activity by ~8%.
- 2 users land in the wrong segment: 4057192912 (Sedentary → actually Low), 6117666160 (Low → actually Somewhat active).
- For a marketing project, wrongly calling people "sedentary" means sending them the wrong message.

Solution

1. **Before deleting**, count each user's non-wear days → `non_wear_days` (goes into the user profile as an **engagement signal** — "how often does this person leave the band at home?").
2. Remove days where `TotalSteps < 100` (covers the 69 zero days + 8 near-zero days).

Option	Rows removed	Recommendation
A · Remove only <code>TotalSteps = 0</code>	69	OK, but leaves 8 obvious non-wear days
B · Remove <code>TotalSteps < 100</code>	77	✓ Recommended

How we'll check

- Minimum `TotalSteps` in clean file ≥ 100
- No rows with 0 calories
- `non_wear_days` column exists for all 33 users (0 for 21 users)

Rows removed: 77 with option B (907 → 830).

P3 add-on · Keep non-wear for the dashboard (Tanay's idea) — ✓ Agreed

Before removing rows, save a separate small table `wear_log` — one row per person per calendar day (33 × 31) with a status: `worn` · `not_worn` (< 100 steps) · `partial` (movement in < 10 hours, see P4) · `last_day` · `stopped` (no row — after the person quit). The analysis tables stay clean; the dashboard's Engagement page uses `wear_log`.

What the data says about WHY people leave the band off (patterns, not proven reasons — there's no survey data):

Possible reason	Evidence	Verdict
Holidays	No user location in data; no major public holidays in 12 Apr–12 May 2016	✗ Can't test / unlikely
Weekends	Weekend 9.5% vs weekday 8.1%; Sunday highest (10.8%) but only 7–13 days per weekday	✗ No clear pattern
Took it off and forgot (e.g., charging)	22 of 35 breaks are a single day . Day before a break: last step at 5 PM (median) vs 11 PM normally; 33% of those days end by 2 PM vs 2% normally. Day after is also short (3,421 steps) → put back on mid-day	✓ Strongest pattern
Novelty wearing off	Non-wear rate week 1: 6.6% → weeks 2–4: 9–10%. 12 users stopped completely before 12 May (all 83 missing calendar days come after they quit)	✓ Supported
Less active people wear it less	On worn days: 6,757 steps (users with non-wear days) vs 9,031 (always-wear); correlation –0.58	✓ Supported
Long breaks (travel / illness / gave up)	A few multi-day breaks, one of 15 days	⚠ Can't tell which

Tanay's battery hypothesis — tested ("0 steps = forgot to charge; 1–100 steps = low battery that ran out"):

Test	Result	What it means
Are zero days really "recorded"?	Each user's zero days show the exact same calories every time (e.g., 1844505072: always 1,347; 6775888955: always 1,841); no heart rate on 38 of 40; effort = resting all day	The system fills an empty day with a fixed resting-calorie estimate and "sitting" minutes. So zero days = no data — dead battery, left at home or left on the charger all look identical. ⚠ Can't prove "forgot to charge"
When do the steps happen on 1–100 step days?	All in one single hour , 6 of 8 between 7 PM and midnight (e.g., 8 steps at 10 PM)	A battery dying <i>during use</i> would show steps from the morning until it died. A single late burst looks like the band being picked up / moved / put on a charger . ✗ Doesn't fit "ran out during use"
Where DOES the battery idea fit?	Day before a break: recording stops early (median 5 PM; 33% by 2 PM vs 2% normally)	Consistent with the battery dying during the day (or being taken off to charge) and not being recharged → next day empty. ✓ Plausible

Decision for labels: dashboard says "Not worn" (with sub-types *no data / briefly handled*), and describes charging as the **likely** cause, not a proven one.

Marketing use: "band off for X hours — put it back on" reminders; battery/charging reminders; re-engagement push around week 2; low-activity users are the priority group for engagement campaigns.

P4 · Band worn only part of the day — ✓ Agreed (10 movement hours)

What's wrong Some days have real steps, but the band was only on for part of the day (e.g., put on at 8 AM, taken off at 1 PM). The rest of the day looks like "no activity", so the day looks much lazier than it was.

Why the original rule (`wear_minutes < 600`) doesn't work — found while checking:

- The system fills missing time with "sitting".** On days without a sleep record, **92%** add up to exactly 1,440 minutes — even when the band was clearly off for most of the day. Example: user 8792009665 on 21 Apr: 144 steps, all at midnight, yet "1,440 minutes recorded". So `wear_minutes` can't see partial wear.
- It catches the wrong days.** Only 2 days fall below 600 minutes, and both are **long-sleep days**, not partial wear: 1844505072 on 30 Apr (402 min + 961 min in bed) and 5553957443 on 30 Apr (590 min + 843 min in bed). Removing them would delete real days.

Better measure: `movement_hours` Count how many of the 24 hours contain **at least 1 step** (from the hourly file). An awake person gets up at least once in almost every hour; a band lying somewhere records

none.

Evidence

Hours with movement	Days	Median steps
1-6	40	1,521
6-8	23	3,414
8-10	25	3,790
10-12	69	6,017
12-24	666	9,146

Median day = 16 movement hours. 28 days have movement that **stops by 2 PM** (e.g., 1927972279 on 7 May: moves only 10-11 AM) and 15 days where movement **starts after 2 PM** — yet 23 of those early-stop days still claim "1,440 minutes recorded". The short days are concentrated in the same people who leave the band off (6775888955: 12 days, 1927972279: 10) → a **wear problem**, not laziness.

Threshold options (after P2 + P3)

Rule: keep days with movement in ≥ ... hours	Rows removed	Users changing activity segment
6	31	3
8	50	4
10	73	6
12	114	8

Solution

- Create `movement_hours` from the hourly file.
- Remove days with **movement_hours < 10** — our version of the common "10 hours of wear" rule.
- **Exception — 7 days where the hourly file is broken:** daily shows real steps (3,008 – 12,015) but all 24 hourly rows show 0 (users 8583815059 ×5, 4319703577 and 4388161847 on 12 Apr). Trust the daily totals → **keep these days**, and exclude them from hourly analysis (see hourly file, H8). Their effort-level breakdown is also missing → handled in P7.
- Mark removed days as `partial` in `wear_log` (P3 add-on) before removing them.
- Drop the idea of `wear_minutes` as a filter.

How we'll check

- Every remaining day has `movement_hours ≥ 10` (or is one of the 7 broken-hourly days)
- Still 33 users
- The long-sleep days of 30 Apr are still present

Rows removed: 73 (830 → 757)

P5 · Sitting time recorded two ways — ✓ Agreed (option B)

What's wrong (simple version) Think of each day as a 1,440-minute pie.

- **Nights when sleep was recorded:** pie = active + sitting + **sleep** → sleep is its own slice.
- **Nights when sleep was NOT recorded:** pie = active + sitting → **sleep gets counted as sitting** (the same "fill empty time as sitting" behaviour found in P3/P4).

Evidence (on the 757 clean days)

	Sleep recorded (387 days)	Sleep not recorded (370 days)
Median sitting time	717 min (12.0 h)	1,157 min (19.3 h)
Median active minutes	267	274
Days adding up to exactly 1,440 min	0%	93%

- On sleep days: active + sitting + time in bed = **exactly 1,440** (median) → the pie theory is confirmed.

- **Same-person proof:** 12 users have both kinds of days. For the *same person*, sitting jumps by **~388 min (6.5 h)** on nights without a sleep record, while active minutes barely change (~11 min). A person doesn't suddenly sit 6.5 hours more — the "extra sitting" is their **sleep**.

Second trap — the "sitting vs sleep" relationship is forced by maths Even on sleep days, sitting and sleep have a -0.69 correlation. That's not a behaviour insight: the pie is fixed at 1,440, so more sleep *must* leave less room for sitting. **We will not present sitting-vs-sleep correlation as a finding.**

Why it matters Raw averages would say people sit 16–20 hours a day, and anyone who didn't track sleep would look like a couch potato.

Options

Option	What	Verdict
A · Flag only	Add <code>sleep_tracked</code> , analysts must remember to filter	OK, but easy to misuse
B · Flag + safe column	Add <code>sleep_tracked</code> and a new column <code>sedentary_awake_minutes</code> = SedentaryMinutes on sleep-tracked days, blank on others. Keep raw column but never chart it	✓ Recommended
C · Guess sleep and subtract it (e.g., -7 h)	Would fill 370 days with invented numbers	✗ Inventing data

Solution (option B)

1. Add `sleep_tracked` (Yes/No) from clean sleepDay.
2. Add `sedentary_awake_minutes` — filled only on sleep-tracked days.
3. **Active minutes** (Very + Fairly + Lightly) and **steps** are our main activity measures — they're unaffected (same-person difference ~11 min).
4. Every sitting-time chart uses `sedentary_awake_minutes` and shows a note: "based on 365 days with a complete night of sleep recorded".
5. Even this column may hide some band-off daytime (filled as sitting) — treat sitting time as the **least reliable** measure.

Coverage after the fix: sitting-time analysis possible for 24 users (16 with 7+ days); 9 users never tracked sleep → no sitting number for them.

How we'll check

- `sleep_tracked` = Yes on **365** of 757 days (updated by sleepDay S5/S6: only **complete, reliable** nights count — 19 short-night days and 3 capped-session days excluded)
- `sedentary_awake_minutes` blank on the other 392
- Median `sedentary_awake_minutes` ≈ 712 (12 h)

P6 · TotalDistance $\neq$ TrackerDistance — ✓ Agreed

What's wrong (simple version) There are three distance columns:

- **TrackerDistance** — what the band measured by itself
- **LoggedActivitiesDistance** — a workout the person **typed into the app** (e.g., "I ran 5 km")
- **TotalDistance** — the final number the app shows = band distance + any part of the typed workout the band **didn't already catch**

They usually match, but differ on 15 days. Which one should we use?

Evidence (757 clean days)

- Only **31 days** have a typed-in workout, from only **3 of 33 users** (9%).
- TotalDistance is **never smaller** than TrackerDistance.
- The difference is **smaller than** the typed distance: e.g., 6962181067 on 9 May typed 3.17 km but TotalDistance is only 0.04 km above TrackerDistance → the band had already recorded most of that workout; the app only added what was missing (no double counting).

- User 8378563200 typed a workout on 16 days (always 2.09 or 2.25 km) and the gap is **0** every time → the band had already caught the whole workout.
- Typed-in distance = only **2%** of all kilometres → tiny effect on distance analysis.

Engagement finding The 3 people who type workouts do it as a **routine**: 7007744171 on 12 days (~4.9 km almost every time), 8378563200 on 16 days (2.09 / 2.25 km), 6962181067 on 3 days. Few users use the feature, but those who do use it regularly — a **feature-adoption opportunity** for marketing.

Solution

1. Use **TotalDistance** as the one distance measure (it's the complete, already de-duplicated number).
2. **Drop TrackerDistance.**
3. **Don't** add LoggedActivitiesDistance to TotalDistance — that would double count.
4. Turn LoggedActivitiesDistance into: - `logged_workout` (day level, Yes/No) - `logs_workouts` + `n_logged_days` (user level, for the engagement page)

Unit note: ~1,465 steps per unit of distance = a normal walking stride in **kilometres**. Label all distances in km.

How we'll check

- TrackerDistance no longer in the table
- `logged_workout` = Yes on 31 days, 3 users

P7 · Effort-level breakdown missing on some days — ✓ Agreed (blank the breakdown)

What's wrong (simple version) Each day has totals (steps, TotalDistance, calories) **and** a breakdown by effort level (very / fairly / light active minutes and distances). The breakdown should add up to the total. On some days it doesn't.

⚠ **Correction:** earlier notes said the cause was hand-logged workouts. Checking the 757 clean days proved that wrong — on logged-workout days the breakdown adds up fine. The real causes are below.

Evidence (17 days where breakdown ≠ total by more than 0.1 km)

Group	Days	What we see	Meaning
A · Breakdown completely missing	7	Real steps (3,008 – 12,015) and distance, but 0 active minutes, 1,440 sitting minutes, 0 km in every effort level . Same 7 days as the broken hourly days (P4 / H8): 8583815059 ×5, 4319703577 and 4388161847 on 12 Apr	The detailed minute data never arrived; only the daily totals survived. "0 active minutes with 12,015 steps" is wrong
B · Breakdown half missing	1	6117666160 on 21 Apr: 19,542 steps, 15.0 km, but effort levels add to only 7.0 km	Part of the day's detail missing; totals look real
C · Small gaps	9	0.1 – 0.8 km differences	Rounding / small sync differences — ignore

Why it matters Group A would add 7 days of "**0 active minutes**" — our **main activity measure** (P5) — to users who actually walked thousands of steps. It would also put 1,440 fake sitting minutes into sitting-time analysis.

Options for group A

Option	What	Verdict
Remove the 7 days	Simple, but deletes real steps/distance/calories	OK
Blank the breakdown	Keep steps, distance, calories. Set the 4 minute columns and 4 effort-distance columns to blank on these 7 days, add flag <code>breakdown_missing</code> = Yes. Averages automatically skip blanks	✓ Recommended — keeps real data, removes wrong data
Leave as is	0 active minutes stays in	✗ Wrong numbers

Solution

1. Group A (7 days): blank the breakdown columns, set `breakdown_missing` = Yes.
2. Group B (1 day): keep as is; use effort-level distances only as **shares**, never as totals.
3. Group C: no action.
4. Drop SedentaryActiveDistance (only 1.46 km in the whole month — see P11).

Useful fact for EDA: share of distance by effort level — **Light 61%, Very active 28%, Moderate 10.5%.**

How we'll check

- 7 days have `breakdown_missing` = Yes and blank minute columns
- No remaining day has steps > 1,000 with 0 active minutes AND 1,440 sitting minutes

P8 · Very high / very low values — ✓ Agreed

What's wrong (simple version) Some days have much bigger numbers than normal (e.g., 36,019 steps). A textbook rule (IQR / "box-plot fence") calls these "outliers" and many tutorials delete them. Should we?

Project rule: we remove a value only if it is **impossible**, never just because it is **unusual**.

Step 1 — Are any values impossible? (757 clean days)

Check	Limit for "impossible"	Days breaking it
Steps	> 50,000	0
Distance	> 60 km	0
Calories	> 6,000	0
Very active minutes	> 300 (5 h hard effort)	0
Calories on a full day	< 1,200	0
Steps per km (stride)	outside ~1,000 – 2,000	0 (range 1,038 – 1,872)

→ **Nothing impossible.** Low steps-per-km days ($\approx 1,040$) belong to runner 8877689391 — running strides are longer, so fewer steps per km. That's physics agreeing with the data.

Step 2 — What the textbook rule would flag

Column	Days flagged "high"	Belong to
TotalSteps (> 20,127)	17	5 users (8877689391 alone: 8)
TotalDistance (> 14 km)	25	6 users
VeryActiveMinutes (> 102)	36	7 users (5577150313: 11, 8053475328: 9)
Calories (> 4,378)	7	3 users
Low side (any column)	0	—

The flagged days are **the same few very active people** doing real workouts. Deleting them would erase the "Very Active" segment — exactly the users marketing wants to understand.

Lowest values are also real: lowest calories = 1,328 (user 1624580081), whose *normal* day is $\sim 1,446$ → a small person with a low resting burn, not an error.

Step 3 — One special day can still mislead a person's average User 1624580081 on 1 May: **36,019 steps, 28 km** (probably an event or long hike) — $4.8\times$ further from their normal than usual. Calories that day (2,690) match a small person walking 28 km, so it's real. But it pulls their **monthly average** from 4,974 (median) up to 5,986 → it would move them from **Sedentary** to **Low active**. 4 users change segment depending on mean vs median.

Solution

1. **Keep every value** — no deletions, no capping.
2. For each person's **typical day** (used for segmentation), use the **median** (the middle day), which one special day can't pull. Show the mean too where helpful (e.g., total effort).

- Charts with long tails (steps, distance) use box plots or medians so a few big days don't squash the rest.
- Note the one odd day: 6117666160 on 21 Apr — highest calories (4,900), 19,542 steps, and half its breakdown missing (P7 group B). Keep, but mention if it shows up in a top-days chart.

How we'll check

- Row count unchanged (757)
- Segments are built from per-person median steps

P9 · Some users have very few clean days — ✓ Agreed

What's wrong (simple version) After cleaning, most people have 3–4 weeks of good days, but a few have only 2–3. Judging someone's "typical day" from 2 days is like rating a restaurant after one bite.

Evidence (clean days per person, after P2-P4)

Clean days	People
1 – 6	2
7 – 14	3
15 – 20	5
21 – 25	4
26 – 31	19

The people at the bottom — and **why** they have so few days:

Person	Days in raw file	Not worn	Partial	Clean days	Story
4057192912	4	1	0	2	Quit after 4 days
6775888955	26	10	12	3	Kept the band 26 days but hardly wore it properly
1927972279	31	13	10	7	Present all month, but only 7 full days
4020332650	31	16	5	9	Same pattern
1844505072	31	12	6	12	Same pattern

Why 7 days as the cut-off? 7 days = one full week, so the person's typical day can include both weekdays and a weekend. The two people below 7 cover only **2 different weekdays** each; 30 of 33 people cover all 7.

Why it matters — two sides

- Activity analysis:** 2–3 days can't describe someone's habits → their "typical day" and segment would be guesswork.
- Engagement analysis:** these are exactly the **low-engagement / drop-out** users marketing needs to see. Throwing them away would hide the problem.

Solution

- Add flag `low_data_user` = Yes for people with **< 7 clean days** (4057192912, 6775888955).
- Exclude** them from per-person activity profiles and activity segments (typical steps, active minutes, sleep profile).
- Keep** them everywhere in day-level charts and in the **engagement analysis** (`wear_log`, non-wear days, drop-outs).
- New user-level column `clean_day_share` = clean days ÷ 31 → feeds an **engagement tier** for the dashboard, e.g.: - **Consistent** (≥ 80% of days clean) - **Irregular** (25–80%) - **Barely using** (< 25% or quit early)

(Exact cut-offs to be fixed in feature engineering.)

How we'll check

- `low_data_user` = Yes for exactly 2 people
- Activity segments cover 31 people; engagement page covers all 33

P10 · Missing calendar days (people who stopped) — ✓ Agreed

What's wrong (simple version) 33 people × 31 days = 1,023 possible days, but the file has only 940. **83 days have no row at all.**

Evidence

- **0 gaps in the middle** of anyone's month. **All 83 missing days come after a person's last day** → these are people who **stopped**, not random skipped days.
- **12 people stopped before 12 May:**

When they stopped	People	Dates
Early (7+ days before the end)	4	15 Apr · 29 Apr · 30 Apr · 1 May
In the final week	8	7 May ×2 · 9 May · 10 May ×2 · 11 May ×3

- ⚠ The final-week group is **uncertain**: 6 of them stopped on 9–11 May, just before the data export. They may simply not have **synced** their band before the export — we can't call them drop-outs with confidence.
- **People counted per week shrinks**: week 1: 33 → week 2: 32 → week 3: 32 → week 4: 29 → last days: 26.

Why it matters

1. **Filling missing days** (with 0 or an average) would invent data — 0 would fake laziness, an average would fake activity.
2. **Hidden trap — changing crowd**: a "steps per week" trend line averages over *whoever is still there*. If less active people leave, the average rises even though **nobody changed**. (People who stopped have typical steps of 7,810 vs 8,475 for those who stayed.) It's like a class average rising because weak students left, not because anyone studied more.

Solution

1. **Never fill** missing days.
2. In `wear_log`, mark them `stopped`.
3. User-level columns: - `last_date`, `days_in_study` - `stopped_early` = Yes if the last day is 7+ days before 12 May (**4 people**) → real drop-outs - `stopped_final_week` = Yes for the 8 uncertain cases (shown separately, not called drop-outs)
4. **Trend charts**: always show "number of users" under each week, **and** draw the main trend line using only people present in every week (so the crowd doesn't change).

How we'll check

- No new rows created (still 757 clean rows)
- `stopped_early` = Yes for 4 people; `stopped_final_week` = Yes for 8

P11 · Tidy the columns + final cleaning order — ✓ Agreed

What's wrong (simple version) Column names are long and mixed-style (`VeryActiveMinutes`, `ModeratelyActiveDistance`), two columns are useless, some numbers carry computer noise (20.39999996185303 km), and the user ID is stored as a number even though it's really a **name tag**.

1 · Drop

Column	Why
TrackerDistance	Incomplete version of TotalDistance (P6)
SedentaryActiveDistance	1.46 km in the whole month — meaningless (P7)

2 · Rename + fix types (one simple naming style: lowercase_with_underscores, units in the name)

Old	New	Type / fix
Id	user_id	Text , not number — nobody should ever add or average IDs
ActivityDate	date	Real date (P1)
TotalSteps	steps	Whole number
TotalDistance	distance_km	Round to 2 decimals
VeryActiveMinutes	very_active_min	Blank on 7 breakdown_missing days (P7)
FairlyActiveMinutes	fairly_active_min	"
LightlyActiveMinutes	lightly_active_min	"
SedentaryMinutes	sedentary_min_raw	"raw" = don't chart directly (P5)
VeryActiveDistance	very_active_km	2 decimals; shares only (P7)
ModeratelyActiveDistance	moderate_active_km	"
LightActiveDistance	light_active_km	"
LoggedActivitiesDistance	logged_km	2 decimals (P6)
Calories	calories	Whole number

3 · Add (day-level helper columns)

New column	How	From
active_min	very + fairly + lightly active minutes	P5 — main measure
movement_hours	hours with ≥ 1 step (hourly file)	P4
day_of_week, is_weekend	from date	Weekday patterns
week_no	1-5 from 12 Apr	Trend charts (P10)
sleep_tracked	Yes if clean sleepDay has that date	P5
sedentary_awake_min	sedentary on sleep-tracked days, blank otherwise	P5
breakdown_missing	Yes on 7 days	P7
logged_workout	logged_km > 0	P6

User-level columns (non_wear_days, clean_day_share, low_data_user, stopped_early, stopped_final_week, logs_workouts, typical/median steps...) do **not** go in the daily table — they go in the **user_profile** table built after all 6 files are cleaned. One table = one level of detail.

4 · Cleaning order (steps depend on each other)

Step	Action	Rows
1	Load; convert dates (P1)	940
2	Compute movement_hours from hourly file	940
3	Build wear_log from the raw rows (33 × 31 grid: worn / not_worn / partial / last_day / stopped) — <i>before</i> anything is removed	—
4	Remove each person's last day (P2)	907
5	Remove days with < 100 steps (P3)	830
6	Remove days with < 10 movement hours, except the 7 broken-hourly days (P4)	757
7	Blank the breakdown on the 7 broken days; add breakdown_missing (P7)	757
8	Add sleep_tracked, sedentary_awake_min — <i>after sleepDay is cleaned</i> (P5)	757
9	Drop / rename / round / add helper columns (P6, P11)	757
10	Save daily_clean.csv and wear_log.csv	—

Final outputs of this file

Table	Rows	Level
daily_clean.csv	757	one person, one day
wear_log.csv	1,023 (33 × 31)	one person, one calendar day — engagement page

How we'll check

- 757 rows, 33 users, no duplicate (user_id, date)
- No column names with capitals; no TrackerDistance / SedentaryActiveDistance
- user_id is text; distances have ≤ 2 decimals
- wear_log status counts: worn 757 · not_worn 77 · partial 73 · last_day 33 · stopped 83 (total 1,023)

Decision log

Date	Decision
2026-09-30	P1 agreed — convert date with explicit month/day/year format, rename to date
2026-09-30	P2 agreed — remove each person's final recorded day (not just 12 May)
2026-09-30	P3 agreed — option B: count non-wear days per user, save wear_log, then remove days with < 100 steps. Charging = likely cause, not proven
2026-09-30	P4 agreed — movement_hours (hours with ≥ 1 step) replaces wear_minutes; keep days with ≥ 10 movement hours; keep 7 broken-hourly days in daily
2026-09-30	P5 agreed — option B: sleep_tracked flag + sedentary_awake_minutes (sleep-tracked days only); active minutes & steps are main measures; no sitting-vs-sleep correlation claims
2026-09-30	P6 agreed — TotalDistance only; drop TrackerDistance; logged distance → logged_workout flags (3 users, 31 days)
2026-09-30	P7 agreed — 7 days with missing effort breakdown: blank the 4 minute + 4 effort-distance columns, flag breakdown_missing; keep steps/distance/calories
2026-09-30	P8 agreed — keep all values (none impossible); per-person typical day = median; charts use medians / box plots
2026-09-30	P9 agreed — low_data_user (< 7 clean days, 2 people): out of activity profiles/segments, kept in engagement analysis; add clean_day_share for engagement tiers
2026-09-30	P10 agreed — never fill missing days; mark stopped in wear_log; stopped_early (4) vs stopped_final_week (8, uncertain); trend charts show users per week + fixed-crowd trend line
2026-09-30	P11 agreed — drop 2 columns, snake_case names with units, user_id as text, 2-decimal km, day-level helpers only (user-level → user_profile), fixed 10-step cleaning order; outputs daily_clean.csv (757) + wear_log.csv (1,023)

02 · Hourly files (hourlySteps + hourlyCalories + hourlyIntensities) — Cleaning Notes

Project: Strava Fitness Data Analytics · **Phase:** 1 (Planning) · **Role:** Time-of-day patterns (when are people active → when to send notifications) · **Status:** ✓ **ALL 9 PROBLEMS AGREED — plan complete**

Running log of problems and solutions. These three files are treated as **one table** because they share exactly the same rows. Many hourly problems are **already decided** by the dailyActivity decisions — they are marked "follows P#".

Files at a glance

Item	Value
What it is	One row = one person, one hour. Steps, calories and effort in that hour
Size	22,099 rows in each of the 3 files
People	33
Time range	12 Apr 2016 00:00 → 12 May 2016 15:00
Blanks / duplicates	None / None
Same rows in all 3 files?	✓ Yes — identical (person, hour) pairs
Replaces	All minute-level steps / calories / intensity files (minutes add up exactly to these hours)
Clock changes (daylight saving) inside the window?	No (US clocks changed 13 Mar 2016 — before the data starts)

Columns we keep / drop

Keep (renamed)	Drop (and why)
Id → user_id (text) · ActivityHour → datetime · StepTotal → steps · Calories → calories · TotalIntensity → intensity	AverageIntensity — exactly TotalIntensity ÷ 60

Problem summary

#	Problem	Solution in one line	Status
H1	Three separate files for one table	Join on user + hour	✓ Agreed
H2	Date-time stored as text	Convert with explicit month/day/year hh:mm:ss AM/PM	✓ Follows P1
H3	Hours from days we removed in daily	Keep only hours of the 750 valid daily days (757 minus 7 broken)	✓ Follows P2-P4
H4	Calories never zero, even with band off	Solved by H3; judge activity by steps/intensity, not calories	✓ Follows P3
H5	Hourly steps don't always add up to daily steps; 4 days miss some hours	Accept small gaps; hourly used only for time-of-day patterns, never for totals	✓ Agreed
H6	Very high hourly values	Keep — nothing impossible	✓ Follows P8
H7	Redundant column + messy names	Drop AverageIntensity; snake_case; add hour/day helpers	✓ Follows P11
H8	7 days where every hour = 0 steps but daily shows thousands	Exclude from hourly only	✓ Follows P4 / P7
H9	Effort with zero steps (e.g., 5 AM workouts)	Keep; flag stepless_effort — shows workouts steps can't see	✓ Agreed

Expected result: 750 valid days → **17,987 hourly rows**, 33 people.

H1 · Three files for one table — ✓ Agreed

Evidence All 3 files have exactly 22,099 rows with the same (person, hour) pairs.

Solution Join on `Id` + `ActivityHour` into one `hourly` table.

How we'll check Joined table has 22,099 rows before filtering — nothing lost or doubled.

H2 · Date-time stored as text — ✓ Follows P1

`ActivityHour` = "4/12/2016 1:00:00 PM" (American format, AM/PM). Convert with the explicit format. Check: 0 failures; 1:00 PM → hour 13; first = 12 Apr 00:00, last = 12 May 15:00.

H3 · Hours from removed days — ✓ Follows P2-P4

Evidence 1,684 hourly rows belong to not-worn days (all 0 steps); plus last days and partial days.

Solution Keep only hours whose (user, date) is in `daily_clean` — **except** the 7 broken days (H8). One rule keeps daily and hourly describing the same days.

Result: 750 days → 17,987 rows (746 days have all 24 hours).

H4 · Calories never zero — ✓ Follows P3

Minimum hourly calories = 42 even with the band off (the system fills in resting calories). Handled by H3. Rule: judge activity by **steps and intensity**, not calories.

H5 · Hourly ≠ daily, and some hours missing — ✓ Agreed

Evidence (750 valid days)

- Hourly steps add up **exactly** to daily steps on **85%** of days. On 95% of days the gap is ≤ 39 steps ($\leq 0.44\%$). Hourly is **never higher** than daily.
- Calories: median gap 0.07%.
- 1 big gap:** 6117666160 on 21 Apr — hourly 9,093 vs daily 19,542 steps. This is the same "half-broken" day from `dailyActivity` P7 (group B).
- 4 days are missing some hour rows** (the rows simply don't exist):

Person	Date	Missing hours
1503960366	11 May	9 PM – 11 PM
6290855005	9 May	5 PM – 11 PM
8253242879	29 Apr	11 PM
8583815059	11 May	10 PM – 11 PM

Why it happens Small syncing differences; the detailed hourly data is occasionally incomplete while the daily total is complete.

Solution

- Daily totals come from `daily_clean`. Hourly is used only for time-of-day patterns** (average steps at each hour, weekday × hour heatmap). Never add hourly up to make daily totals.
- Don't fill missing hours (no inventing). For "average steps at hour X", a missing hour simply isn't counted.
- Keep 6117666160 on 21 Apr in hourly (its hours look normal; only the total is higher).

How we'll check No chart or number computes daily totals from the hourly table.

H6 · Very high hourly values — ✓ Follows P8

Max 10,554 steps in one hour (user 8877689391, 30 Apr 2 PM, intensity 180 = maximum possible, 948 calories) — a full hour of running by the known runner. Only 1 hour exceeds 10,000. Nothing impossible → keep.

H7 · Columns — ✓ Follows P11

Drop: AverageIntensity. **Rename:** snake_case (user_id, datetime, steps, calories, intensity).

Add:

New column	Why
date	Link to daily_clean
hour (0-23)	Time-of-day charts → best notification times
day_of_week, is_weekend	Weekday × hour heatmap
stepless_effort	H9

H8 · Broken hourly days — ✓ Follows P4 / P7

Evidence 7 days: all 24 hours = 0 steps while daily shows 3,008-12,015 steps (8583815059 ×5; 4319703577 and 4388161847 on 12 Apr). Their daily effort breakdown is also blank.

Solution Excluded from hourly (not in the 750 days). Kept in daily for steps / distance / calories (P7).

H9 · Effort with zero steps — ✓ Agreed

What's wrong (simple version) Some hours show effort (intensity) but **0 steps**. Normally effort comes from walking/running, so how?

Evidence

- 183 such hours. 162 are tiny (intensity 1-5 → noise, e.g., arm movement).
- **18 are big** (intensity 96-165, 290-670 calories in one hour) — real workouts with no steps.
- **16 of the 18 = user 8378563200 at 5 AM**, on exactly the days they typed in a workout (dailyActivity P6: 2.09 / 2.25 km). Intensity 165, 669 calories, 0 steps — every time.
- → These are **workouts the step counter can't see** — e.g., cycling, swimming, gym machines — added through the typed-in workout.
- The other 2 big ones (2873212765 at 6 AM, 6290855005 at 9 PM) are on days **without** a typed-in workout → the band itself felt hard effort with no steps.
- Hour-by-hour example (8378563200, 13 Apr): 4 AM → 306 steps, intensity 10 · **5 AM → 0 steps, intensity 165** · 6 AM → 175 steps, intensity 6.
- What kind of workout? About 2 km in an hour of very hard effort fits **swimming or a gym machine** better than cycling (cycling usually covers far more km per hour) — the data can't say for sure.

Why it matters (Strava angle!) Step-based measures **under-count** people whose workouts aren't walking/running. This is the closest the data gets to Strava's "cycling / other workouts".

- User 8378563200 ranks **16th of 33 by steps** (middle of the pack) but **4th of 33 by very-active minutes**.
- On their workout days: 78 very-active minutes vs 7.5 on other days.
- → A **steps-only** segmentation would call them "average"; by effort they are among the most active.
- → Segmentation must use **both steps and active minutes** (decision for feature engineering).

Solution

- Keep all these hours (real effort).
- Flag `stepless_effort` = Yes when steps = 0 and intensity ≥ 30 (the 18 real ones; ignore the tiny noise).

- In EDA, mention that step counts miss non-walking workouts; use **active minutes / intensity** when comparing workout effort.

How we'll check `stepless_effort` = Yes on 18 hours (16 from 8378563200).

Decision log

Date	Decision
2026-09-30	H2, H3, H4, H6, H7, H8 follow the agreed dailyActivity decisions (P1–P4, P7, P8, P11)
2026-09-30	H1 agreed — join the 3 hourly files on user + hour
2026-09-30	H5 agreed — hourly used only for time-of-day patterns, never for daily totals; missing hours not filled
2026-09-30	H9 agreed — keep step-free effort hours; flag <code>stepless_effort</code> (steps = 0 & intensity ≥ 30, 18 hours); segmentation must use steps AND active minutes

03 · sleepDay — Cleaning Notes

Project: Strava Fitness Data Analytics · **Phase:** 1 (Planning) · **Role:** Nightly sleep totals · **Status:** ✓ **ALL 9 PROBLEMS AGREED — plan complete**

Running log of problems and solutions.

File at a glance

Item	Value
What it is	One row = one person, one sleep date. Minutes asleep, minutes in bed, number of sleeps
Size	413 rows × 5 columns
People	24 (all of them also in dailyActivity)
Dates	12 Apr 2016 → 12 May 2016
Blanks	None
Exact duplicate rows	3

Columns we keep

Keep (renamed)	Notes
Id · SleepDay → <code>date</code> · TotalSleepRecords → <code>sleep_records</code> · TotalMinutesAsleep → <code>minutes_asleep</code> · TotalTimeInBed → <code>minutes_in_bed</code>	All 5 columns are useful — nothing to drop

Problem summary

#	Problem	Solution in one line	Status
S1	Exact duplicate rows	Remove duplicates	✓ Agreed
S2	Date stored as text (with a fake 12:00 AM time)	Convert, keep date only	✓ Follows P1
S3	The date means "the morning you woke up"	Document it; shift date when linking to that day's activity	✓ Agreed
S4	Naps are mixed into the night totals	Rebuild night sleep from minuteSleep: main sleep + pieces within 2 h = night; the rest = naps	✓ Agreed
S5	Very short "nights" (< 3 hours)	Flag <code>incomplete_night</code> ; daytime-only sessions become naps; exclude from nightly averages AND from <code>sleep_tracked</code>	✓ Agreed
S6	Very long sleeps	4 sessions stuck at exactly 961 min = recording glitch → flag <code>capped_session</code> , exclude like S5; other long nights (10-12 h) are real → keep, flag <code>long_sleep</code>	✓ Agreed
S7	One user is restless in bed a lot	Keep; flag <code>restless_sleeper</code> (user level); use medians; no health labels	✓ Agreed
S8	Users with very few reliable nights	Flag <code>low_sleep_data_user</code> (< 7 reliable nights, 8 people); 16 people get sleep profiles; add sleep-feature adoption tier (all 33)	✓ Agreed
S9	Final sleep table + helper columns	One <code>sleep_nights.csv</code> (410 rows) built from minuteSleep + sleepDay; user-level items → <code>user_profile</code>	✓ Agreed

S1 · Exact duplicate rows — ✓ Agreed

Evidence 3 rows are exact copies:

- 4388161847 on 5 May (471 asleep / 495 in bed)
- 4702921684 on 7 May (520 / 543)
- 8378563200 on 25 Apr (388 / 402)

Why it matters Those nights would count twice in averages, and would create doubled rows when joined to dailyActivity.

Solution Remove exact duplicates → **410 rows**.

How we'll check No (Id, date) appears twice.

S2 · Date stored as text — ✓ Follows P1

What's wrong SleepDay = "4/12/2016 12:00:00 AM". The time is always 12:00 AM — it carries no information.

Solution Convert with month/day/year hour:min:sec AM/PM, then keep only the date. Rename to date.

How we'll check 0 failures; joins with clean dailyActivity on (Id, date).

S3 · The date means "the morning you woke up" — ✓ Agreed

What's wrong It's not obvious which night a date refers to. A sleep from 11 PM on 11 Apr to 7 AM on 12 Apr — is it "11 Apr" or "12 Apr"?

Evidence Rebuilding nights from minuteSleep: using the **wake-up date**, time in bed matches sleepDay on **100%** of rows. Using the **start date**, only **37%** match. → SleepDay = the date you **woke up**.

Why it matters When we join sleep to daily activity on the same date, we get "**last night's sleep → today's activity**". For the question "**does today's activity affect tonight's sleep?**" we need the *next* date's sleep.

Solution

- Keep date as the wake-up date (documented).
- When building the daily master table, add two views:
 - sleep_last_night = sleep with the same date
 - sleep_tonight = sleep with date + 1 day
- Choose the view based on the question being asked.

Linkage numbers (clean data): 387 clean daily days have "last night's sleep"; 380 have "tonight's sleep".

Sleep is cleaned on its own rules. Removing a daily row (e.g., a partial day) does **not** remove that morning's sleep — the night was recorded fine. 23 of the 410 nights have no matching clean daily day; they stay in **sleep-only** analysis and simply have no activity to compare with.

Example of how one date works (user 1503960366, 13 Apr): main sleep 3:08 AM → 8:20 AM **and** a nap 8:10 PM → 9:43 PM. Both carry the date 13 Apr (TotalSleepRecords = 2). → see S4 for separating naps.

S4 · Naps are mixed into the night totals — ✓ Agreed

What's wrong (simple version) sleepDay gives one total per date. If a person slept at night **and** napped in the afternoon, both are added together. So "8 hours of sleep" might really be 5 hours at night + a 3-hour nap.

Evidence (410 nights)

- 364 nights = 1 sleep session, **43 = 2 sessions, 3 = 3 sessions** → 49 extra sessions.
- On the 46 multi-session dates, total sleep median = **451 min**, but the main sleep alone = **316 min** → totals inflated by ~2 hours.

- Using minuteSleep, the 49 extra sessions are of **two different kinds**:

Kind	Count	Example	What it really is
Piece of a broken night — starts/ends within 2 h of the main sleep	19	1503960366: slept 2:11–6:59 AM, then again 7:02–8:19 AM	Woke up briefly and went back to bed → part of the night
Real nap — separate from the night	30 (9 people)	3977333714: night 1:04–8:30 AM, nap 3:20–4:34 PM	A nap

- Naps are 60–252 min (median ~2 h), mostly afternoons (1–4 PM) and some evenings (8–10 PM, dozing before bed). 4388161847 naps often (sometimes twice a day).

Why it matters

- "Average nightly sleep" would be inflated by naps.
- Treating a broken night's second piece as a "nap" would make restless sleepers look like nappers.
- Bedtime would be wrong if a nap were taken as the start of the night.

Solution (built from minuteSleep — see file 04, M5)

- For each person + wake date: **main sleep** = the longest session.
- Any other session starting or ending **within 2 hours** of the main sleep → **also night sleep** (broken night).
- Everything else → **nap**.
- New columns per night: - `night_minutes_asleep`, `night_minutes_in_bed` (main + broken-night pieces) - `nap_minutes`, `took_nap` (Yes/No), `night_pieces` (1 = unbroken night)
- Keep sleepDay's original totals as `total_asleep_incl_naps` / `total_in_bed_incl_naps` for reference.
- All **nightly** sleep charts use `night_minutes_asleep`.

How we'll check

- night + nap minutes = sleepDay totals (time in bed matches 100%, as verified in S3)
- 30 nap sessions from 9 people; 19 broken-night pieces

S5 · Very short "nights" (< 3 hours) — ✓ Agreed

What's wrong (simple version) After S4, 22 dates still have less than 3 hours of night sleep (as low as 58 min). Real adults rarely sleep under 3 hours — more likely **the night wasn't fully recorded**.

Evidence — two kinds

Kind	Count	Examples	What it really is
A · Only a daytime sleep recorded (starts 11 AM – 7:49 PM)	10	7007744171 on 1 May: 11:29 AM – 12:29 PM · 1644430081 on 29 Apr: 6:33 – 8:39 PM	A nap ; the real night was never recorded
B · Night-time start, but cut short	12	4445114986 on 17 Apr: 2:24 – 4:10 AM · 4558609924: 10:41 PM – 12:41 AM (and 4 more nights like it) · a few 8–10 PM dozes ending before 11 PM	Band removed / battery died mid-night, or an evening doze with the night unrecorded

- 4558609924 has 5 of the 22 → a recording habit, not a sleep problem.
- Knock-on effect on dailyActivity P5:** on these days the unrecorded sleep is counted as sitting again — median sitting **1,096 min** vs **712 min** on complete-night days. So these days must NOT count as `sleep_tracked`.

Night-sleep distribution (all 410 dates)

< 3 h	3–5 h	5–6 h	6–7 h	7–9 h	> 9 h
22	36	52	77	188	35

Solution

1. Flag `incomplete_night` = Yes when night sleep < 180 min (22 dates).
2. Kind A: move that session to **nap** (night sleep = blank for that date).
3. Exclude all 22 from nightly averages, sleep categories and bedtime/wake-time stats. Keep them in the data (they still count as "sleep feature used").
4. **Update dailyActivity P5:** `sleep_tracked` = Yes only when a **complete** night is recorded → **368** of 757 clean days (was 387); after S6 capped sessions → **365**. `sedentary_awake_min` uses these days only.

How we'll check

- 22 dates flagged; 388 complete nights remain for sleep analysis
- Median sitting on `sleep_tracked` days ≈ 712 min

S6 · Very long sleeps — ✓ Agreed

What's wrong (simple version) Some nights show 10–13 hours of sleep. Real (a lie-in after a hard week) or a recording problem?

Evidence (night sleep after S4, 410 dates)

- 35 nights > 9 h · **13 nights > 10 h** · 2 nights > 12 h.
- 🚩 **4 sessions last exactly 961 minutes (16 h 01 min)** — same length to the minute, from 2 different people:

Person	Start	End	Asleep
1644430081	1 May 6:01 PM	2 May 10:01 AM	796
1844505072	14 Apr 8:16 PM	15 Apr 12:16 PM	644
1844505072	29 Apr 10:09 PM	30 Apr 2:09 PM	722
1844505072	30 Apr 8:52 PM	1 May 12:52 PM	590

Natural sleep never stops at exactly the same minute 4 times → looks like a **recording limit** (the sleep recording kept running until it hit a 16-hour maximum, e.g., sleep mode left on). Lying still in bed gets counted as "asleep", so the minutes are unreliable. 1844505072's typical "night" (644 min) is driven by these.

The other 10 long nights (10–11.7 h) look real:

- **6 of 10 are weekend mornings** (Sat/Sun) — lie-ins. (Weekends are only 2 of 7 days.)
- Several follow **very active days**: 5553957443 slept 11 h after 12,764 steps, 10.9 h after 15,482, 10.8 h after 16,556 → recovery sleep.
- One odd case: 5577150313 on 4 May slept **10:30 AM - 9:03 PM** (daytime) — sick day or shift work; keep.

Solution

1. Flag `capped_session` = Yes for sessions of exactly 961 min (4 dates) → treat like S5: exclude from sleep-duration averages, categories and bedtime/wake stats; not counted as `sleep_tracked` in daily.
2. Flag `long_sleep` = Yes for other nights > 10 h (10 dates) → **keep** in all analysis (real behaviour).
3. EDA ideas noted: weekend lie-ins; recovery sleep after very active days.

How we'll check

- 4 `capped_session` dates, 10 `long_sleep` dates
- No session of exactly 961 min left in duration stats

S7 · One user is restless in bed a lot — ✓ Agreed

What's wrong (simple version) Sleep efficiency = time asleep ÷ time in bed. Most people are asleep for ~94% of their time in bed. One person is far below everyone else — error or real?

Evidence (384 reliable nights — after removing S5 short nights and S6 capped sessions)

- Median efficiency for everyone: **94%**. Common benchmark for "good" sleep efficiency: **85%+**.

- 28 nights are below 85% and 23 below 70% — and **23 of those 23 belong to one person, 3977333714.**

Person	Reliable nights	Efficiency (median)	Minutes in bed not asleep (median)
3977333714	27	63%	162 (155 restless + 4 awake)
Next lowest (2347167796)	15	90%	49
Typical person	—	94-95%	~25

- It's **consistent**: 23 of their 27 nights are below 70% (range 53-78%). Not one bad night — a pattern.
- It's **spread across the whole night**: 30-43% of every quarter of the night is restless, vs 5-8% for everyone else. So it's not "takes long to fall asleep" — the whole night is broken.
- Mostly marked **restless** (level 2), not fully **awake** (level 3).

Real or device? We can't tell for sure — it could be genuinely restless sleep, or the band sensing more movement (e.g., worn loosely). Either way it's consistent, so it's **not a random error**.

Why it matters

- Deleting them would hide a real sleep profile — useful for sleep-feature marketing ("improve your sleep quality").
- Keeping them could drag group **averages** down → use **medians** (as agreed in P8).

Solution

1. **Keep** all their nights.
2. Flag `restless_sleeper` = Yes at user level (median efficiency < 85% → only 3977333714).
3. Group sleep-quality numbers use **medians**.
4. Use neutral wording only ("restless sleep pattern") — **no health or medical labels**; the data can't diagnose anything.
5. New per-night columns: `sleep_efficiency`, `restless_minutes`, `awake_minutes` (from `minuteSleep`).

How we'll check Group median efficiency ≈ 94% with or without this person; flag = Yes for 1 person.

Side-finding for S8: after S5 + S6, only **19 of 24** sleep users have at least one reliable night. 5 people (1644430081, 1844505072, 2320127002, 4558609924, 7007744171) have **only** short or capped nights. (Corrected from an earlier count of 20/4.)

S8 · Users with very few reliable nights — ✓ Agreed

What's wrong (simple version) Same idea as dailyActivity P9: you can't describe someone's sleep habits from 2 nights. And after S5/S6, some people have **no** reliable night at all.

Evidence — reliable nights per person (24 sleep users)

Reliable nights	People	Who
0 (only short / capped nights)	5	1644430081, 1844505072, 2320127002, 4558609924, 7007744171
1 - 6	3	8053475328 (2), 6775888955 (3), 1927972279 (4)
7 - 19	4	4020332650 (7), 8792009665 (15), 2347167796 (15), 6117666160 (18)
20+	12	the rest

Sleep-feature adoption story (all 33 people) — a ready-made engagement chart:

Level	People	Share
Never tracked sleep	9	27%
Tried it, but < 7 reliable nights	8	24%
Regular (7-19 reliable nights)	4	12%
Consistent (20+ reliable nights)	12	36%

→ Only about **half** of users get real value from sleep tracking. Marketing angle: help the "tried it" group turn it into a habit.

Solution

1. Flag `low_sleep_data_user` = Yes for **< 7 reliable nights** (8 people) — same 7-day rule as P9.
2. **Per-person sleep profiles** (typical sleep hours, efficiency, bedtime) → only the **16** people with 7+ reliable nights.
3. **Night-level charts** keep every reliable night from everyone (19 people).
4. Add `sleep_adoption_level` (never / tried / regular / consistent) to user_profile for the engagement page.

How we'll check 16 people with sleep profiles; adoption levels add up to 33.

S9 · Final sleep table + helper columns — ✓ Agreed

Idea (simple version) sleepDay and minuteSleep describe the **same nights** at two levels of detail. Instead of two half-useful tables, we build **one clean night table**: the detail (bedtime, naps, restlessness) comes from minuteSleep, the original totals from sleepDay as a cross-check.

Output: `sleep_nights.csv` — one row per person per wake-up date (410 rows, 24 people)

Column	Meaning	From
<code>user_id</code> (text), <code>date</code>	person, wake-up date	S2, S3
<code>total_asleep_incl_naps</code> , <code>total_in_bed_incl_naps</code> , <code>sleep_records</code>	sleepDay's original numbers (reference only)	S4
<code>night_minutes_asleep</code> , <code>night_minutes_in_bed</code>	real night sleep (main + broken-night pieces)	S4
<code>night_hours_asleep</code>	÷ 60, easy to read	S9
<code>sleep_efficiency</code>	asleep ÷ in bed	S7
<code>minutes_awake_in_bed</code> , <code>restless_minutes</code> , <code>awake_minutes</code>	quality detail	S7
<code>night_pieces</code>	1 = slept straight through	S4
<code>bedtime</code> , <code>wake_time</code> (clock time)	start of first / end of last night piece	S4
<code>bedtime_hrs_after_6pm</code>	e.g., 11:30 PM → 5.5, 12:30 AM → 6.5 (so averages work across midnight)	M6
<code>nap_minutes</code> , <code>took_nap</code>	naps that day	S4, S5
<code>incomplete_night</code> , <code>capped_session</code> , <code>long_sleep</code>	flags	S5, S6
<code>reliable_night</code>	Yes if not incomplete and not capped (384 nights)	S5, S6
<code>sleep_category</code>	< 6 h · 6–7 h · 7–9 h (recommended) · > 9 h — reliable nights only	S9
<code>day_of_week</code> , <code>is_weekend_morning</code>	weekend lie-in analysis	S6

Goes to user_profile (not this table): `low_sleep_data_user`, `restless_sleeper`, `sleep_adoption_level`, typical sleep hours / efficiency / bedtime (medians, 16 people).

Goes to daily master (when joining): `sleep_last_night` / `sleep_tonight` (S3), `sleep_tracked` = reliable night on that date (365 clean days).

How we'll check

- 410 rows, 24 people, no duplicate (user_id, date)
- night + nap minutes = sleepDay totals (time in bed 100% match)

- 384 reliable nights; `sleep_category` blank on the other 26
- Median night sleep ≈ 7 h 13 min (433 min); median efficiency $\approx 94\%$; median bedtime $\approx 11:13$ PM
- Sleep categories (reliable nights): < 6 h: 86 · 6–7 h: 78 · 7–9 h: 189 · > 9 h: 31

Decision log

Date	Decision
2026-09-30	S1 agreed — remove 3 exact duplicates (413 → 410)
2026-09-30	S2 follows P1 — explicit format, keep date only
2026-09-30	S3 agreed — <code>date</code> = wake-up date; daily master gets <code>sleep_last_night</code> (same date) and <code>sleep_tonight</code> (next date); sleep cleaned independently of daily removals
2026-09-30	S4 agreed — <code>night</code> = main session + pieces within 2 h; others = naps; new <code>night_minutes_asleep</code> , <code>nap_minutes</code> , <code>took_nap</code> , <code>night_pieces</code> ; <code>sleepDay</code> totals kept as <i>incl_naps</i>
2026-09-30	S5 agreed — <code>incomplete_night</code> (< 180 min, 22 dates); daytime-only sessions → naps; excluded from nightly stats; <code>sleep_tracked</code> in daily only for complete nights (368 days)
2026-09-30	S6 agreed — 4 sessions of exactly 961 min flagged <code>capped_session</code> and excluded from duration stats (like S5); other > 10 h nights kept as <code>long_sleep</code> ; <code>sleep_tracked</code> in daily → 365 days
2026-09-30	S7 agreed — keep 3977333714's nights; user flag <code>restless_sleeper</code> (median efficiency $< 85\%$); medians for group sleep quality; neutral wording, no health labels
2026-09-30	S8 agreed — <code>low_sleep_data_user</code> (< 7 reliable nights, 8 people); sleep profiles for 16 people; night charts use all reliable nights (19 people); <code>sleep_adoption_level</code> for all 33
2026-09-30	S9 agreed — one <code>sleep_nights.csv</code> (410 rows, 24 people) built from <code>minuteSleep</code> detail + <code>sleepDay</code> totals; bedtime as hours after 6 PM; user-level sleep items → <code>user_profile</code>

04 · minuteSleep — Cleaning Notes

Project: Strava Fitness Data Analytics · **Phase:** 1 (Planning) · **Role:** Bedtime, wake time, naps, restlessness
· **Status:** ✓ **ALL 8 PROBLEMS AGREED — plan complete**

Running log of problems and solutions. We use this file only to **derive** sleep-timing columns — its minute rows are summarised to one row per sleep session, then one row per night.

File at a glance

Item	Value
What it is	One row = one minute of one sleep session, marked 1 = asleep, 2 = restless, 3 = awake
Size	188,521 rows × 4 columns
People	24 (same people as sleepDay)
Time range	11 Apr 2016 20:48 → 12 May 2016 09:56
Sleep sessions (logId)	459 — exactly equal to the total of sleepDay's TotalSleepRecords ✓
Blanks	None
Exact duplicate rows	543

Columns

Keep (renamed)	Meaning
Id · date → datetime · value → sleep_state · logId → session_id	All 4 needed

Problem summary

#	Problem	Solution in one line	Status
M1	Exact duplicate rows (one whole night copied twice)	Remove duplicates	✓ Agreed
M2	Date-time stored as text; some times end in :30 seconds	Convert; round down to the minute	✓ Agreed
M3	Codes 1/2/3 are not readable	Map to asleep / restless / awake	✓ Agreed
M4	Sessions cross midnight	Assign each session to its wake-up date	✓ Follows S3
M5	Naps vs main sleep not labelled	Main = longest session per wake date; sessions within 2 h of it = same night; rest = naps	✓ Follows S4
M6	Minute-level data too detailed for analysis	Summarise to session, then to night → sleep_nights.csv	✓ Follows S9
M7	4 sessions last exactly 961 min (recording limit)	Flag capped_session ; exclude from duration stats (see sleepDay S6)	✓ Follows S6
M8	Minutes asleep differ slightly from sleepDay on 14 nights	Use minuteSleep as the source; accept small gaps	✓ Agreed

M1 · Exact duplicate rows — ✓ Agreed

Evidence 543 rows are exact copies. They are **one entire night copied twice**: user 4702921684, session 11573168523, 6 May 9:10 PM → 7 May 6:12 AM (1,086 rows = 543 × 2). The **same night is also duplicated in sleepDay** (S1: 4702921684 on 7 May) → one export glitch hit both files. No case where the same minute has two *different* values.

Solution Remove exact duplicates → 187,978 rows.

How we'll check No (Id, datetime) appears twice; that session's length now matches sleepDay's time in bed.

M2 · Text date-time and :30 seconds — ✓ Agreed

Evidence 126,714 times end in :00 and 61,807 end in :30 (e.g., "2:47:30 AM"). Each **session is consistent** — 159 of 459 sessions are entirely at :30, none are mixed. So the :30 is just where that session's recording started.

Solution Convert with the explicit format (follows P1), then round **down** to the whole minute. Safe: after rounding, **0** duplicate minutes and **0** overlapping sessions.

M3 · Codes not readable — ✓ Agreed

Evidence Values: 1 = 172,480 min, 2 = 14,023 min, 3 = 2,018 min.

Solution Map 1 → asleep, 2 → restless, 3 → awake.

M4 · Sessions cross midnight — ✓ Follows S3

Evidence 254 of 459 sessions start on one date and end on the next (e.g., 11 PM → 7 AM).

Solution Give each session a wake_date = date of its last minute. This matches sleepDay exactly (time in bed matches **100%** using wake date vs 37% using start date — see sleepDay S3).

M5 · Naps vs main sleep — ✓ Follows S4

Evidence Session start times cluster at 8 PM–3 AM (the normal night), but **39 sessions start between 9 AM and 7 PM** (naps). Shortest session = 60 minutes; no tiny fragments.

Solution

- For each person + wake_date: the **longest** session = main_sleep.
- Another session starting/ending **within 2 hours** of the main sleep = night_piece (broken night — woke up and went back to bed). **19** such pieces.
- Everything else = nap. **30** naps from 9 people.
- **Bedtime** = start of the earliest night session; **wake time** = end of the latest night session. Naps never set bedtime.

How we'll check 459 sessions = 410 main + 19 night pieces + 30 naps; bedtimes mostly fall in the evening/night. (See sleepDay S4 for the evidence.)

M6 · Summarise to a usable level — ✓ Follows S9

Step A — one row per session

Column	How
sleep_start, sleep_end	first / last minute
minutes_asleep, minutes_restless, minutes_awake	count of each state
session_type	main_sleep / nap (M5)

Step B — one row per person per night (wake_date)

Column	Why it matters
bedtime (clock time of main sleep start)	Sleep timing pattern; late-sleeper segment
wake_time (clock time of main sleep end)	Morning routine → best time for morning notifications

Column	Why it matters
restless_minutes, awake_minutes	Sleep quality beyond sleepDay's totals
nap_minutes, took_nap	Nap behaviour

Bedtime tip for later: a bedtime of 11:30 PM and one of 12:30 AM are only 1 hour apart, but as clock numbers they are 23.5 and 0.5. For averages, measure bedtime as "hours after 6 PM" (11:30 PM → 5.5, 12:30 AM → 6.5).

Result: joins onto the daily master table using (Id, wake_date).

M8 · Small differences vs sleepDay — ✓ Agreed

Evidence Time in bed matches sleepDay on **100%** of nights. Minutes asleep match on **96.6%**; on the other 14 nights minuteSleep counts **1-22 more** asleep minutes (median 4).

Solution minuteSleep is our source for night sleep (it's the detailed record). Keep sleepDay totals only as reference. A 4-minute gap changes nothing.

Decision log

Date	Decision
2026-09-30	M4-M7 follow sleepDay S3, S4, S9, S6
2026-09-30	M1, M2, M3, M8 agreed — drop the duplicated night; explicit format + round down to minute; map 1/2/3 → asleep/restless/awake; minuteSleep is the source for night sleep

05 · weightLogInfo — Cleaning Notes

Project: Strava Fitness Data Analytics · **Phase:** 1 (Planning) · **Role:** BMI profile + feature adoption · **Status:**  **ALL 7 PROBLEMS AGREED — plan complete**

Running log of problems and solutions.

File at a glance

Item	Value
What it is	One row = one weight entry: weight, BMI, body fat, typed by hand or smart scale
Size	67 rows × 8 columns
People	8 of 33 (24%)
Dates	12 Apr 2016 → 12 May 2016
Duplicate rows / same person twice on one day	None / None
Blanks	Fat: 65 of 67 empty

Columns we keep / drop

Keep (renamed)	Drop (and why)
Id · Date → <code>date</code> · WeightKg → <code>weight_kg</code> · BMI → <code>bmi</code> · IsManualReport → <code>is_manual</code>	Fat — 97% empty · WeightPounds — just kg × 2.2046 · LogId — a record number, not unique across people

Problem summary

#	Problem	Solution in one line	Status
W1	Fat column almost empty	Drop it	 Agreed
W2	Duplicate information / ID columns	Drop WeightPounds and LogId	 Agreed
W3	Date-time stored as text	Convert; keep date only	 Follows P1
W4	Very uneven entries per person	Summarise to one row per person (median)	 Agreed
W5	One very high BMI (47.5)	Keep — consistent and possible; use median + categories	 Agreed
W6	Only 8 people	Use as descriptive profile + feature-adoption signal only; no BMI-vs-activity claims	 Agreed
W7	Final tables + helper columns	<code>weight_logs_clean.csv</code> (67) + <code>user_body_profile</code> (8); WHO BMI categories; <code>logs_weight</code> , <code>features_used</code> for all 33	 Agreed

W1 · Fat column almost empty — Agreed

Evidence Only 2 of 67 entries have a Fat value: 1503960366 on 2 May (22%) and 4319703577 on 17 Apr (25%) — both typed by hand, once each.

Solution Drop. Two values can't support any analysis, and filling 65 blanks would be inventing data.

W2 · Duplicate information / ID columns — ✓ Agreed

Evidence

- $\text{WeightPounds} = \text{WeightKg} \times 2.20462$ on every row.
- LogId is **not unique**: it is simply the entry's **date-time written as milliseconds since 1970** (e.g., 1462233599000 = 2 May 2016 11:59:59 PM). Manual entries are all stamped 11:59:59 PM, so two people typing on the same date get the same LogId — 10 LogIds are shared between people. It adds nothing the Date column doesn't already have.

Solution Drop both. We use kg (standard for India/most of the world).

W3 · Date-time stored as text — ✓ Follows P1

Evidence All 41 manual entries are stamped "11:59:59 PM" (the person entered only a date). Smart-scale entries carry the real time — mostly **6:39-6:51 AM** → a morning weigh-in routine.

Solution Convert with month/day/year hour:min:sec AM/PM, keep date only → date .

W4 · Very uneven entries per person — ✓ Agreed

Evidence

Person	Entries	Avg BMI	Avg kg	Entry type
6962181067	30	24.0	61.6	Manual
8877689391	24	25.5	85.1	Smart scale
4558609924	5	27.2	69.6	Manual
1503960366	2	22.6	52.6	Manual
2873212765	2	21.6	57.0	Manual
4319703577	2	27.4	72.4	Manual
1927972279	1	47.5	133.5	Smart scale
5577150313	1	28.0	90.7	Smart scale

Two people = 54 of 67 rows (81%). Weight barely changes in a month (max change 1.8 kg).

Why it matters Averaging all rows would mostly describe 2 people.

Solution

1. Keep the cleaned entry-level table (`weight_logs_clean.csv` , 67 rows) for a simple "weight over time" view of the 2 regular loggers.
2. Summarise to **one row per person** (`user_body_profile` , 8 rows): - `bmi` and `weight_kg` = **median** of their entries (same rule as P8; weight changes ≤ 1.8 kg in the month, so median $\approx$ latest) - `n_weight_logs` , `manual_share` , `first_log_date` , `last_log_date`
3. Every person counts **once** in BMI charts — no matter whether they logged 1 time or 30.

How we'll check `user_body_profile` has 8 rows; the two heavy loggers count once each.

W5 · One very high BMI — ✓ Agreed

Evidence User 1927972279: 133.5 kg, BMI 47.5. Check: implied height = $\sqrt{(\text{kg} \div \text{BMI})}$ = **1.68 m**, a normal height. Implied height is constant for every person across all their entries → BMI values are internally consistent.

More checks

- Recorded **automatically by a smart scale** (`IsManualReport` = False) → not a typing mistake.
- Implied heights for all 8 people range 1.52 – 1.83 m and stay **constant** across every person's entries → BMI is calculated consistently.

- It fits the rest of this person's data: lowest typical steps of anyone ($\approx 2,163$ on clean days), 13 not-worn days, 7 clean days.

Effect on group numbers (8 people)

- **Mean** BMI = 28.0 — pulled up by this one person
- **Median** BMI = 26.4 — not affected

Solution

1. **Keep.** Possible, consistent, machine-recorded — not an error.
2. Group BMI = **median** + **category counts** (W7), never the plain mean.
3. Neutral wording only (use the standard category names; no personal remarks).

W6 · Only 8 people — ✓ Agreed

What's wrong (simple version) Only 8 of 33 people ever logged their weight. With 8 people, any "BMI affects activity" conclusion would be a guess — one person changes the whole picture.

What we CAN and CAN'T say

✓ Can say (descriptive / engagement)	✗ Can't say (too few people)
BMI mix of those who log: 3 normal, 4 overweight, 1 obese (median BMI 26.4)	"Higher BMI → fewer steps" or any BMI-vs-activity relationship
Only 24% of users ever logged weight	"Overweight users sleep less" etc.
Only 2 of 8 built a habit (30 and 24 entries); 6 logged 1–5 times and stopped	Anything about BMI for the other 25 users
5 of 8 typed by hand, 3 used a smart scale; scale users weigh in at ~6:45 AM	
Weight loggers look slightly more engaged: typical 8,863 steps vs 8,053; 28.5 clean days vs 26 (directional only)	

Bigger engagement idea found while checking — "feature depth" Counting how many *optional* features each person uses (sleep tracking, weight logging, workout logging):

Optional features used	People	Typical steps	Typical clean days
0	7	7,396	24
1	18	8,409	28
2	7	8,596	27
3	1	10,479	30

→ People who use **more features** tend to be **more active and wear the band more**. Small numbers and not cause-and-effect (active people may simply like more features), but a strong marketing story: *get users to adopt a second feature*.

Solution

1. Use weight/BMI only for: (a) the descriptive BMI mix, (b) feature-adoption numbers. Every chart states "**n = 8 users**".
2. **No** BMI-vs-activity or BMI-vs-sleep claims.
3. Add `features_used` (0–3) to `user_profile` for the engagement page.

W7 · Final tables + helper columns — ✓ Agreed

Output 1 · `weight_logs_clean.csv` — one row per entry (67 rows, 8 people)

Column	Notes
<code>user_id</code> (text), <code>date</code>	W3

Column	Notes
weight_kg	1 decimal (raw has noise like 52.599998)
bmi	1 decimal
is_manual	Yes = typed by hand, No = smart scale
bmi_category	see below

Output 2 • user_body_profile — one row per person (8 rows)

Column	How
bmi, weight_kg	median of their entries (W4)
bmi_category	from median BMI
n_weight_logs	count of entries
manual_share	share typed by hand
first_log_date, last_log_date	first / last entry
weight_habit	Yes if ≥ 10 entries (the 2 regular loggers)

BMI categories (WHO international standard)

BMI	Category	People
< 18.5	Underweight	0
18.5 - 24.9	Normal	3
25 - 29.9	Overweight	4
≥ 30	Obese	1

Note: India/Asia often uses lower cut-offs (23 / 27.5). The data doesn't say where users live, so we use the WHO international standard and state it on the chart.

Goes to user_profile (all 33 people)

- logs_weight (Yes/No) — 8 Yes / 25 No
- features_used (0-3) = sleep tracking + weight logging + workout logging (W6)
- plus the 8 rows of user_body_profile joined in (blank for the other 25)

How we'll check

- 67 entry rows, 8 profile rows, no Fat / WeightPounds / LogId columns
- Category counts 0 / 3 / 4 / 1; features_used counts 7 / 18 / 7 / 1

Decision log

Date	Decision
2026-09-30	W1, W2 agreed — drop Fat (97% empty), WeightPounds (duplicate), LogId (date-time in ms, not unique)
2026-09-30	W3 follows P1 — explicit format, keep date only
2026-09-30	W4 agreed — one row per person (user_body_profile , 8 rows) using median BMI/weight; entry list kept for over-time view
2026-09-30	W5 agreed — keep BMI 47.5 (smart scale, consistent height); group BMI = median (26.4) + category counts; neutral wording
2026-09-30	W6 agreed — weight/BMI used only for descriptive BMI mix + feature adoption (n = 8 on every chart); no BMI-vs-activity claims; add features_used
2026-09-30	W7 agreed — outputs weight_logs_clean.csv (67) + user_body_profile (8, median BMI, WHO categories stated on chart); logs_weight + features_used for all 33

06 · heartrate_seconds — Cleaning Notes

Project: Strava Fitness Data Analytics · **Phase:** 1 (Planning) · **Role:** Optional supporting evidence (effort & fitness) · **Status:** ✓ **ALL 7 PROBLEMS AGREED — plan complete**

Running log of problems and solutions. This is the biggest file; we **summarise it to one row per person per day** and never use the raw readings in the dashboard.

File at a glance

Item	Value
What it is	Heart beats per minute, recorded every ~5 seconds
Size	2,483,658 rows × 3 columns (~90 MB)
People	14 of 33
Time range	12 Apr 2016 → 12 May 2016 16:20
Blanks / duplicates	None / None
Value range	36 – 203 bpm (median 73)

Columns

Keep (renamed)	
Id → <code>user_id</code> (text) · Time → <code>datetime</code> · Value → <code>heart_rate</code>	All 3 needed

Problem summary

#	Problem	Solution in one line	Status
R1	Too big and too detailed	Summarise to one row per person per day (<code>daily_heart_rate</code>)	✓ Agreed
R2	Date-time stored as text	Explicit format (also makes loading much faster)	✓ Follows P1
R3	Uneven coverage — band off for hours	Keep only days with ≥ 10 hours (600 min) of readings	✓ Agreed
R4	Extreme readings (> 200, < 40)	Nothing impossible → keep; use percentiles (5th / 99th) instead of min / max	✓ Agreed
R5	Days that are invalid in <code>dailyActivity</code>	Keep only days in <code>daily_clean</code>	✓ Follows P2–P4
R6	People with little heart-rate data	Flag <code>low_hr_data_user</code> (< 7 valid days)	✓ Agreed
R7	What can we honestly claim?	Use as sanity check + effort detail; don't claim "active → lower resting HR"	✓ Agreed

Expected result: 334 person-days → **280 valid days**, 13 people (12 with HR profiles).

R1 · Too big and too detailed — ✓ Agreed

Why it matters 2.5 million rows would make a Streamlit dashboard very slow, and nobody needs 5-second detail for a marketing decision.

Solution Summarise to one row per person per day (`daily_heart_rate.csv`):

Column	How	Meaning
<code>hr_avg</code>	mean	Typical heart rate that day
<code>hr_resting_est</code>	5th percentile	Stand-in for resting heart rate

Column	How	Meaning
hr_peak	99th percentile	Hardest effort that day (robust — R4)
minutes_hr_above_100	minutes whose average HR > 100	Time in elevated effort
hr_minutes_recorded	minutes with any reading	Coverage (R3)

Check of the resting estimate: median 5th percentile = **59 bpm**; median heart rate **during recorded sleep** (11 people) = **60.8 bpm** → the estimate matches real resting heart rate well.

Result: 334 rows instead of 2.5 million → ~280 after R3/R5.

R2 · Date-time stored as text — ✓ Follows P1

Explicit month/day/year hh:mm:ss AM/PM. For 2.5 M rows, giving the format also makes loading **much** faster than letting Python guess.

R3 · Uneven coverage — ✓ Agreed

Evidence

- Readings usually every 5 s (90% within 15 s), but **612 gaps > 10 minutes** (band off).
- Minutes with readings per person-day: median 974 (~16 h); **50 of 334** days have < 10 h; one has 4 minutes.
- **2026352035** has 4 days of readings and **none** reaches 10 hours → drops out entirely.

Why it matters A day with 2 hours of readings (e.g., only a workout) gives a misleading daily average.

Solution Keep only days with `hr_minutes_recorded ≥ 600` (same 10-hour idea as P4) → **284** days.

R4 · Extreme readings — ✓ Agreed

Evidence

- **> 200 bpm:** 13 readings, all user 2022484408 on 21 Apr at 4:31 PM. Heart rate goes 92 → 200 in 5 s, stays 180–203 for ~12 minutes, then slowly recovers (154 → 131 → 122). **Steps = 0** in those minutes, but the hourly effort that hour is high (intensity 97 with only 506 steps) — the same "effort without steps" pattern as hourly H9 (e.g., a bike). ⚠ **Correction:** earlier notes called this "probably a sensor glitch". It's **uncertain**: the instant jump looks like a glitch, the slow recovery looks real. 203 bpm is not impossible for a young adult.
- **< 40 bpm:** 23 readings across 3 users, mostly 2–7 AM during sleep (possible for fit people); a few mid-afternoon.
- Together: 36 of 2.48 million readings (0.001%).
- Daily max is > 30 bpm above the 99th percentile on 41 days → single spikes are common; max/min are unreliable.

Solution

- **Keep** all readings (nothing impossible; can't prove the 21 Apr episode is fake).
- Use **percentiles** for daily summaries: 5th for resting, 99th for peak — a few seconds of spikes can't move them.
- Note 21 Apr (2022484408) as a "check this" day if it appears in a top-peaks chart.

R5 · Invalid days — ✓ Follows P2-P4

Keep only (user, date) pairs in `daily_clean` → **280** days (284 with coverage n clean daily).

R6 · People with little heart-rate data — ✓ Agreed

Evidence — valid days per person (after R3 + R5)

Person	Valid days
2026352035	0 (dropped by R3)
6775888955	3
4020332650	8
Others (11)	16 – 30

Solution Flag `low_hr_data_user` (< 7 valid days: 2026352035, 6775888955). Per-person HR profiles → **12 people**; day-level charts keep all 280 days. Every HR chart states "n = 12–13 users".

R7 · What can we honestly claim? — ✓ Agreed

Evidence (280 valid days)

- ✓ **Sanity check passes:** very-active minutes vs minutes with HR > 100 → correlation **0.5**; steps vs average HR → **0.44**. The band's effort data and heart data agree.
- ✗ **"More active people have a lower resting heart rate" is NOT supported here:** across the 13 people, resting estimate vs typical steps → correlation **+0.22** (wrong direction, and 13 people is too few anyway). E.g., the lowest resting HR (5577150313: 48.5) has average steps; 7007744171 has high steps and the highest resting HR (74).

Solution — how the file is used

1. **Data-quality proof** in the EDA: "effort minutes line up with heart rate" (builds trust in the activity data).
2. **Effort detail** on the dashboard: minutes in elevated heart rate per day, for the 12 people.
3. **No** fitness claims about resting heart rate vs activity.

Decision log

Date	Decision
2026-09-30	R2 follows P1; R5 follows P2–P4
2026-09-30	R1 agreed — summarise to <code>daily_heart_rate.csv</code> (avg, 5th pct resting, 99th pct peak, minutes > 100, coverage)
2026-09-30	R3 agreed — keep days with ≥ 600 minutes of readings
2026-09-30	R4 agreed — keep all readings; 21 Apr episode (2022484408) uncertain, marked 'check this'; daily summaries use 5th / 99th percentiles, never min / max
2026-09-30	R6 agreed — <code>low_hr_data_user</code> (< 7 valid days: 2026352035, 6775888955); HR profiles for 12 people
2026-09-30	R7 agreed — HR used as data-quality proof + effort detail; no resting-HR-vs-activity claims